

WHY BACKTRACKING?

Backtracking is a controlled DFS search over a **State Space Tree**. It explores candidate solutions and **backtracks** (undoes choice) as soon as a path violates constraints!

THE 3-STEP TRINITY

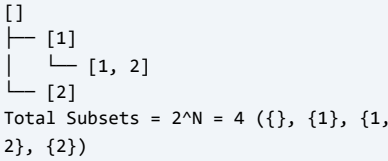
- 1. **CHOOSE:** Add candidate to path (`list.add(val)`).
- 2. **EXPLORE:** Recurse to next level (`dfs(...)`).
- 3. **UNDO:** Remove candidate (`list.remove(list.size() - 1)`).

3 CORE CATEGORIES

- **Subsets (2^N):** Pick or skip each element ('start' pointer increments).
- **Combinations ($N! / K!$):** Choose K elements out of N ('start' pointer).
- **Permutations ($N!$):** Order matters ('visited' array or element swapping).

2 VISUALIZING STATE SPACE TREE

Subsets of [1, 2]:



PREREQUISITES & COMPLEXITY REASONING

Category	Time Complexity	Auxiliary Space
Subsets (2^N)	$O(N \cdot 2^N)$	$O(N)$ recursion stack
Combinations (nCr)	$O(K \cdot nCr)$	$O(K)$ stack
Permutations ($N!$)	$O(N \cdot N!)$	$O(N)$ stack
N-Queens / Sudoku	$O(N!)$	$O(N)$ board state

CLUES THAT SCREAM BACKTRACKING

- 1. "Return all possible combinations / subsets / permutations"
- 2. "Duplicates allowed in input, output must contain **UNIQUE** results"
→ Sort + Skip duplicate adjacent elements
- 3. "Place N queens on board / solve Sudoku"
→ Constraint Validation Grid DFS

1 UNIVERSAL SUBSETS TEMPLATE

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    backtrack(res, new ArrayList<>(), nums, 0);
    return res;
}

private void backtrack(List<List<Integer>> res, List<Integer> track, int[] nums, int start) {
    res.add(new ArrayList<>(track)); // Snapshot copy!
    for (int i = start; i < nums.length; i++) {
        track.add(nums[i]);           // 1. Choose
        backtrack(res, track, nums, i + 1); // 2. Explore
        track.remove(track.size() - 1); // 3. Undo
    }
}
```

2 DUPLICATE PRUNING PATTERN

```
// MANDATORY: Sort array first!
Arrays.sort(nums);

for (int i = start; i < nums.length; i++) {
    // Prune duplicates at same recursion depth:
    if (i > start && nums[i] == nums[i - 1]) continue;
    track.add(nums[i]);
    backtrack(res, track, nums, i + 1);
    track.remove(track.size() - 1);
}
```

'i > start' ensures we allow duplicate values at DIFFERENT recursion levels, but skip duplicates at SAME level!

PATTERN 1SUBSETS I (Power Set Generation)e.g. Subsets (LC 78)	
WHEN TO USE Generate all 2^N subsets. Every node in the state space tree is a valid subset snapshot!	TEMPLATE — LC 78 <pre>public List<List<Integer>> subsets(int[] nums) { List<List<Integer>> res = new ArrayList<>(); dfs(res, new ArrayList<>(), nums, 0); return res; } private void dfs(List<List<Integer>> res, List<Integer> track, int[] nums, int start) { res.add(new ArrayList<>(track)); for (int i = start; i < nums.length; i++) { track.add(nums[i]); dfs(res, track, nums, i + 1); track.remove(track.size() - 1); } }</pre>
PATTERN 2SUBSETS II (Duplicates in Input)e.g. Subsets II (LC 90)	
WHEN TO USE Input array contains duplicates. Must sort and skip duplicate branch at same level (<code>i > start</code>).	TEMPLATE — LC 90 <pre>public List<List<Integer>> subsetsWithDup(int[] nums) { Arrays.sort(nums); List<List<Integer>> res = new ArrayList<>(); dfs(res, new ArrayList<>(), nums, 0); return res; } private void dfs(List<List<Integer>> res, List<Integer> track, int[] nums, int start) { res.add(new ArrayList<>(track)); for (int i = start; i < nums.length; i++) { if (i > start && nums[i] == nums[i - 1]) continue; track.add(nums[i]); dfs(res, track, nums, i + 1); track.remove(track.size() - 1); } }</pre>

PATTERN 3 COMBINATION SUM I (Unlimited Reuse of Elements) e.g. Combination Sum (LC 39)

UNLIMITED REUSE TRICK

Pass `i` (NOT `i + 1`) to next recursive call so current element can be chosen repeatedly!

TEMPLATE — LC 39

```
public List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> res = new ArrayList<>();
    dfs(res, new ArrayList<>(), candidates, target, 0);
    return res;
}

private void dfs(List<List<Integer>> res, List<Integer> track, int[] nums, int remain, int start) {
    if (remain < 0) return;
    if (remain == 0) { res.add(new ArrayList<>(track)); return; }
    for (int i = start; i < nums.length; i++) {
        track.add(nums[i]);
        dfs(res, track, nums, remain - nums[i], i); // Pass 'i' for reuse!
        track.remove(track.size() - 1);
    }
}
```

PATTERN 4 COMBINATION SUM II (Single Use per Element + Duplicates) e.g. Combination Sum II (LC 40)

SINGLE USE + DUP PRUNE

Sort array first. Pass `i + 1` for single use. Skip duplicates (`i > start && nums[i] == nums[i-1]`).

TEMPLATE — LC 40

```
public List<List<Integer>> combinationSum2(int[] candidates, int target) {
    Arrays.sort(candidates);
    List<List<Integer>> res = new ArrayList<>();
    dfs(res, new ArrayList<>(), candidates, target, 0);
    return res;
}

private void dfs(List<List<Integer>> res, List<Integer> track, int[] nums, int remain, int start) {
    if (remain < 0) return;
    if (remain == 0) { res.add(new ArrayList<>(track)); return; }
    for (int i = start; i < nums.length; i++) {
        if (i > start && nums[i] == nums[i - 1]) continue;
        track.add(nums[i]);
        dfs(res, track, nums, remain - nums[i], i + 1);
        track.remove(track.size() - 1);
    }
}
```

PATTERN 5 PERMUTATIONS I (Order Matters, Distinct Elements) e.g. Permutations (LC 46)**VISITED TRACKING**

Order matters, so loop from `0` to `N-1` at every level. Track used elements via `boolean[] visited` or `track.contains()`.

TEMPLATE — LC 46

```
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    dfs(res, new ArrayList<>(), nums, new boolean[nums.length]);
    return res;
}

private void dfs(List<List<Integer>> res, List<Integer> track, int[] nums, boolean[] vis) {
    if (track.size() == nums.length) { res.add(new ArrayList<>(track)); return; }
    for (int i = 0; i < nums.length; i++) {
        if (vis[i]) continue;
        vis[i] = true; track.add(nums[i]);
        dfs(res, track, nums, vis);
        track.remove(track.size() - 1); vis[i] = false;
    }
}
```

PATTERN 6 PERMUTATIONS II (Order Matters + Input Duplicates) e.g. Permutations II (LC 47)**VISITED PRUNING CONDITION**

Sort array first. Skip duplicate if `nums[i] == nums[i-1]` AND `!visited[i-1]`.

TEMPLATE — LC 47

```
public List<List<Integer>> permuteUnique(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();
    dfs(res, new ArrayList<>(), nums, new boolean[nums.length]);
    return res;
}

private void dfs(List<List<Integer>> res, List<Integer> track, int[] nums, boolean[] vis) {
    if (track.size() == nums.length) { res.add(new ArrayList<>(track)); return; }
    for (int i = 0; i < nums.length; i++) {
        if (vis[i] || (i > 0 && nums[i] == nums[i - 1] && !vis[i - 1])) continue;
        vis[i] = true; track.add(nums[i]);
        dfs(res, track, nums, vis);
        track.remove(track.size() - 1); vis[i] = false;
    }
}
```

PATTERN 7

WORD SEARCH IN 2D GRID (Grid DFS Backtracking)

e.g. Word Search (LC 79)

GRID BACKTRACKING STATE

Temporarily replace `board[r][c] = '#'` to mark visited, then restore original character after child DFS calls return!

TEMPLATE — LC 79

```
public boolean exist(char[][] board, String word) {
    for (int r = 0; r < board.length; r++)
        for (int c = 0; c < board[0].length; c++)
            if (dfs(board, word, r, c, 0)) return true;
    return false;
}
private boolean dfs(char[][] b, String w, int r, int c, int idx) {
    if (idx == w.length()) return true;
    if (r<0||r>=b.length||c<0||c>=b[0].length||b[r][c] != w.charAt(idx)) return false;
    char tmp = b[r][c]; b[r][c] = '#'; // Mark
    for (int[] d : DIRS) {
        if (dfs(b, w, r + d[0], c + d[1], idx + 1)) return true;
    }
    b[r][c] = tmp; // Backtrack!
    return false;
}
```

PATTERN 8

N-QUEENS (Row-by-Row Placement Validation)

e.g. N-Queens (LC 51)

CONSTRAINT VALIDATION

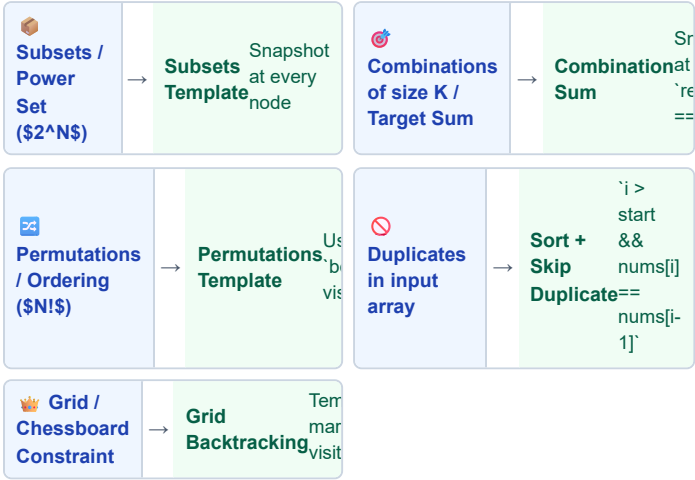
Place queen row by row. Track valid columns, main diagonals ('r - c'), and anti-diagonals ('r + c').

TEMPLATE — LC 51

```
public List<List<String>> solveNQueens(int n) {
    List<List<String>> res = new ArrayList<>();
    char[][] board = new char[n][n];
    for (char[] row : board) Arrays.fill(row, '.');
    dfs(res, board, 0, n); return res;
}
private void dfs(List<List<String>> res, char[][] b, int row, int n) {
    if (row == n) { res.add(construct(b)); return; }
    for (int col = 0; col < n; col++) {
        if (isValid(b, row, col, n)) {
            b[row][col] = 'Q'; dfs(res, b, row + 1, n); b[row][col] = '.';
        }
    }
}
```

BACKTRACKING — WHICH PATTERN TO USE?

Start: What is the output structure?



TRIGGER WORDS DIRECTORY

Trigger Word	Variant	Action
"All possible subsets"	Subsets I/II	Snapshot list at start of DFS
"Combinations summing to target"	Combination Sum	Pass <code>`remain - nums[i]`</code>
"All arrangements / orderings"	Permutations	Loop 0..N-1 with <code>`visited[]`</code>
"Unique results only"	Duplicate Pruning	Sort array + skip adjacent dups

INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
Why pass <code>`i + 1`</code> vs <code>`i`</code> in DFS?	Pass <code>`i + 1`</code> when each element can be used AT MOST ONCE. Pass <code>`i`</code> when element can be reused infinitely!
Why sort first for duplicates?	Sorting brings identical elements adjacent so <code>`nums[i] == nums[i-1]`</code> can skip duplicate search subtrees!

EASY & MEDIUM — Subsets & Combinations

LC 78 · Subsets

MEDIUM

Pattern: Subsets I
Key Insight: Snapshot copy at every node.

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    dfs(res, new ArrayList<>(), nums, 0);
    return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] n, int s) {
    res.add(new ArrayList<>(trk));
    for (int i = s; i < n.length; i++) {
        trk.add(n[i]);
        dfs(res, trk, n, i + 1);
        trk.remove(trk.size() - 1);
    }
}
```

LC 90 · Subsets II

MEDIUM

Pattern: Subsets II Dup Pruning
Key Insight: Sort + `i > start && nums[i] == nums[i-1]`.

```
public List<List<Integer>> subsetsWithDup(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();
    dfs(res, new ArrayList<>(), nums, 0);
    return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] n, int s) {
    res.add(new ArrayList<>(trk));
    for (int i = s; i < n.length; i++) {
        if (i > s && n[i] == n[i - 1]) continue;
        trk.add(n[i]);
        dfs(res, trk, n, i + 1);
        trk.remove(trk.size() - 1);
    }
}
```

LC 39 · Combination Sum

MEDIUM

Pattern: Unlimited Element Reuse
Key Insight: Pass `i` to next recursive call.

```
public List<List<Integer>> combinationSum(int[] cand, int target) {
    List<List<Integer>> res = new ArrayList<>();
    dfs(res, new ArrayList<>(), cand, target, 0);
    return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] c, int rem, int s) {
    if (rem < 0) return;
    if (rem == 0) { res.add(new ArrayList<>(trk)); return; }
    for (int i = s; i < c.length; i++) {
        trk.add(c[i]);
        dfs(res, trk, c, rem - c[i], i);
        trk.remove(trk.size() - 1);
    }
}
```

MEDIUM — Permutations & Single Use Combo

LC 40 · Combination Sum II

MEDIUM

Pattern: Single Use + Dup Pruning
Key Insight: Sort + `i + 1` + skip duplicate.

```
public List<List<Integer>> combinationSum2(int[] cand, int target) {
    Arrays.sort(cand);
    List<List<Integer>> res = new ArrayList<>();
    dfs(res, new ArrayList<>(), cand, target, 0);
    return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] c, int rem, int s) {
    if (rem < 0) return;
    if (rem == 0) { res.add(new ArrayList<>(trk)); return; }
    for (int i = s; i < c.length; i++) {
        if (i > s && c[i] == c[i - 1]) continue;
        trk.add(c[i]);
        dfs(res, trk, c, rem - c[i], i + 1);
        trk.remove(trk.size() - 1);
    }
}
```

LC 46 · Permutations

MEDIUM

Pattern: Visited Boolean Permutations
Key Insight: Loop 0..N-1, skip if `visited[i]`.

```
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    dfs(res, new ArrayList<>(), nums, new boolean[nums.length]);
    return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] n, boolean[] vis) {
    if (trk.size() == n.length) { res.add(new ArrayList<>(trk)); return; }
    for (int i = 0; i < n.length; i++) {
        if (vis[i]) continue;
        vis[i] = true;
        trk.add(n[i]);
        dfs(res, trk, n, vis);
        trk.remove(trk.size() - 1);
        vis[i] = false;
    }
}
```


MEDIUM — Grid & Palindrome Partitioning

LC 79 · Word Search

MEDIUM

Pattern: Grid DFS Backtracking

Key Insight: Temp replace `board[r][c] = '#'`.

```
private boolean dfs(char[][] b, String w, int r, int c, int idx) {
    if (idx == w.length()) return true;
    if (r<0||r>=b.length||c<0||c>=b[0].length||
        b[r][c] != w.charAt(idx)) return false;
    char tmp = b[r][c]; b[r][c] = '#';
    for (int[] d : DIRS) if (dfs(b, w, r + d[0], c + d[1], idx + 1)) return true;
    b[r][c] = tmp; return false;
}
```

LC 131 · Palindrome Partitioning

MEDIUM

Pattern: Substring Partitioning

Key Insight: Recurse if `isPalindrome(s, start, i)`.

```
private void dfs(List<List<String>> res, List<String> trk, String s, int start) {
    if (start == s.length()) { res.add(new ArrayList<>(trk)); return; }
    for (int i = start; i < s.length(); i++) {
        if (isPal(s, start, i)) {
            trk.add(s.substring(start, i + 1));
            dfs(res, trk, s, i + 1);
            trk.remove(trk.size() - 1);
        }
    }
}
```

HARD — N-Queens Board Placement

LC 51 · N-Queens

HARD

Pattern: Constraint Grid DFS

Key Insight: Validate row, column, and diagonals.

```
private void dfs(List<List<String>> res, char[][] b, int row, int n) {
    if (row == n) { res.add(construct(b)); return; }
    for (int col = 0; col < n; col++) {
        if (isValid(b, row, col, n)) {
            b[row][col] = 'Q'; dfs(res, b, row + 1, n); b[row][col] = '.';
        }
    }
}
```

LC 17 · Phone Number Combinations

MEDIUM

Pattern: Digit Keypad Mapping

Key Insight: Map digits to letters string array.

```
private void dfs(List<String> res, StringBuilder sb, String digits, int idx, String[] map) {
    if (idx == digits.length()) { res.add(sb.toString()); return; }
    String letters = map[digits.charAt(idx) - '0'];
    for (char c : letters.toCharArray()) {
        sb.append(c); dfs(res, sb, digits, idx + 1, map); sb.deleteCharAt(sb.length() - 1);
    }
}
```

COMPLETE BACKTRACKING PROBLEM LADDER SUMMARY					
LC #	Problem	Pattern	Time	Space	Diff
78	Subsets	Subsets I	$O(N \cdot 2^N)$	$O(N)$	M
90	Subsets II	Subsets II Dup Pruning	$O(N \cdot 2^N)$	$O(N)$	M
39	Combination Sum	Unlimited Reuse	$O(2^T)$	$O(T)$	M
40	Combination Sum II	Single Use + Dup Pruning	$O(2^N)$	$O(N)$	M
46	Permutations	Visited Permutations	$O(N \cdot N!)$	$O(N)$	M
47	Permutations II	Permutations Dup Pruning	$O(N \cdot N!)$	$O(N)$	M
79	Word Search	Grid DFS Backtracking	$O(N \cdot 4^L)$	$O(L)$	M
131	Palindrome Partitioning	Substring Partitioning	$O(N \cdot 2^N)$	$O(N)$	M
51	N-Queens	Constraint Grid DFS	$O(N!)$	$O(N^2)$	H

DRY RUN — Subsets ([1, 2])

Step	Track State	Action	Snapshot Added to Result
1	[]	<code>`res.add([])`</code>	<code>`[]`</code>
2	[1]	Add 1, recurse <code>`start=1`</code> $\rightarrow$ <code>`res.add([1])`</code>	<code>`[1]`</code>
3	[1, 2]	Add 2, recurse <code>`start=2`</code> $\rightarrow$ <code>`res.add([1, 2])`</code>	<code>`[1, 2]`</code>
4	[1]	Pop 2 $\rightarrow$ Return to level 1	-
5	[]	Pop 1 $\rightarrow$ Loop <code>`i=1`</code> (add 2) $\rightarrow$ <code>`res.add([2])`</code>	<code>`[2]`</code> ✓

MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Write Subsets I in 3m
- ☐ Code Subsets II duplicate pruning
- ☐ Write Combination Sum (element reuse `i`)
- ☐ Code Permutations with ``boolean[] visited``
- ☐ Write Permutations II duplicate check
- ☐ Code Word Search 2D Grid DFS
- ☐ Write N-Queens board placement DFS
- ☐ Prove why Subsets is $O(N \cdot 2^N)$

MATHEMATICAL PROOF — 2^N SUBSETS DERIVATION

Claim: An array of N distinct elements produces exactly 2^N subsets.

Proof: For each element $x_i \in [1..N]$, we have binary choices: INCLUDE x_i or EXCLUDE x_i . Total subsets $= 2 \times 2 \times \dots \times 2 = 2^N$. Snapshot copy takes $O(N)$ time $\rightarrow$ **$O(N \cdot 2^N)$ Time!**

GOLDEN RULES — NEVER FORGET

- Rule 1 — Always Make a Copy for Result**

Always add a NEW copy to result list (``res.add(new ArrayList<>(track))``), NOT the original reference ``res.add(track)``!
- Rule 2 — Always Undo the Last Element**

After the recursive call returns, immediately undo the choice: ``track.remove(track.size() - 1)``!